COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	14/10/26	**DATE SUBMITTED:**	16/10/26
NAME:	Cruz, Elijah Villaruel, Prince Benjamin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	Tree and Graph Traversal	Visits nodes by calling the same function repeatedly until all needed nodes are visited or a stopping condition is reached.
Brute Force	Linear Search	Check each item one by one until the target is found or the list ends. It is simple but can take longer for large lists.
Backtracking	Depth-First Search (DFS)	Explores one path as deeply as possible. If the path does not work or reaches a dead end, it goes back and tries another path.
Greedy	Selection Sort	Repeatedly selects the smallest item from the unsorted part and places it in the correct position until the list is sorted.
Divide-and-Conquer	Merge Sort	Divides the list into smaller parts, sorts each part, and merges them back together in the correct order.

Table 12-1

CODE SNIPPET

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

// array to store answers
std::vector<int> memo;

int getMinSteps(int n) {
    if (n == 1) return 0; // base case
    if (memo[n] != -1) return memo[n]; // check if saved    no operator "[]" matches these operands

    // option 1: subtract 1
    int r = 1 + getMinSteps(n - 1);

    // option 2: divide by 2
    if (n % 2 == 0) r = std::min(r, 1 + getMinSteps(n / 2));

    // option 3: divide by 3
    if (n % 3 == 0) r = std::min(r, 1 + getMinSteps(n / 3));

    memo[n] = r; // save result    no operator "[]" matches these operands
    return r;
}

void test(int n) {
    memo.assign(n + 1, -1); // reset array with -1    class "std::vector<int, std::allocator<int>>" has no member "assign"
    std::cout << getMinSteps(n) << std::endl;
}

int main() {
    test(6);
    test(8);
    test(11);
    test(10);

    return 0;
}
```

CODE OUTPUT

```
PS C:\Users\TIPQC\Documents\PAIR CPE21S2>
2
3
4
3
```

CODE ANALYSIS

We followed the solution provided in the laboratory guidelines by defining our base case as 0 when n = 1 and choosing the path with the fewest steps among subtracting 1, dividing by 2, and dividing by 3. We preserved the same recursive approach and structure, ensuring we look up previously saved results before making an extra function call. We switched from a raw array to std::vector, which allows us to reset our memory table with -1 before each run. Lastly, we wrapped our test execution in a small helper function to display outputs clearly and keep the main function short.

Table 12-2

CODE SNIPPET

MAIN.CPP 1

MAIN.CPP > main()

```cpp
#include <iostream>
#include <algorithm>
using std::min;

int getMinSteps ( int n )
{
    int dp[n+1], i;    expression must have a constant value
    dp[1] = 0; // trivial case
    for( i = 2 ; i <= n ; i ++ )
    {
        dp[i] = 1 + dp[i-1];
        if(i%2==0) dp[i] = min( dp[i] , 1+ dp[i/2] );
        if(i%3==0) dp[i] = min( dp[i] , 1+ dp[i/3] );
    }
    return dp[n];
}

int main()
{
    int tests[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 50, 100};
    for ( int n : tests )
        std::cout << n << "\t" << getMinSteps( n ) << "\n";
    return 0;
}
```

CODE OUTPUT

```
PS C:\Users\TIPQC\Documents\cruz>
1       0
2       1
3       1
4       2
5       3
6       2
7       3
8       3
9       2
10      3
15      4
20      4
50      6
100     7
PS C:\Users\TIPQC\Documents\cruz>
```

CODE ANALYSIS

As observed, the function finds the fewest steps needed to turn a number `n` into 1. In each step you can subtract 1, divide by 2 (if it divides evenly), or divides by 3. It works by solving small numbers first and saving each answer in an array called `dp`. Then it starts with `dp[1] = 0`, because 1 does not need steps. Then, for each number from 2 up to `n`, it picks the cheapest option: subtract 1, or divide by 2 or 3 when possible. It uses the saved answers of the smaller numbers, so nothing is calculated twice. For example, to reach 1 from 10, the best path is 10 to 9 to 3 to 1, which takes 3 steps.

Table 12-3

COMPUTER ENGINEERING

QUEZON CITY

SUPPLEMENTARY ACTIVITY

Copy and paste the questions from the lab and include your answers after.
ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

PSEUDOCODE

```
START
        INPUT M, N
        INPUT matrix mat
        INPUT cost

        result = countPaths(M - 1, N - 1, cost)
        OUTPUT result

END

FUNCTION countPaths(i, j, cost)
        IF i < 0 OR j < 0 OR cost < 0 THEN
                RETURN 0
        IF i == 0 AND j == 0 THEN
                IF cost == mat[0][0] THEN
                        RETURN 1
                ELSE
                        RETURN 0
        RETURN countPaths(i - 1, j, cost - mat[i][j]) + countPaths(i, j - 1, cost - mat[i][j])

END FUNCTION
```

CODE SNIPPET

main.cpp

main.cpp > main()

```cpp
#include <iostream>

int mat[100][100];   // the matrix

// counts the paths from (0,0) to (i,j) that have the given cost
int countPaths(int i, int j, int cost) {

    // outside the matrix or cost is already negative
    if (i < 0 || j < 0 || cost < 0)
        return 0;

    // reached the first cell
    if (i == 0 && j == 0) {
        if (cost == mat[0][0])
            return 1;   // the path has the exact cost
        else
            return 0;
    }

    // a cell can be reached from the top or from the left
    return countPaths(i - 1, j, cost - mat[i][j]) + // subtract the current cell's value from the cost
           countPaths(i, j - 1, cost - mat[i][j]);
}

int main() {
    int M, N, cost;

    std::cout << "Enter rows and columns: ";
    std::cin >> M >> N;

    // input the matrix
    std::cout << "Enter the matrix:\n";
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            std::cin >> mat[i][j];
        }
    }

    std::cout << "Enter target cost: ";
    std::cin >> cost;

    // start from the last cell (M-1, N-1)
    int result = countPaths(M - 1, N - 1, cost);

    std::cout << "Number of paths with cost " << cost << " = " << result << std::endl;

    return 0;
}
```

COMPUTER ENGINEERING

QUEZON CITY

CODE OUTPUT

1.

```
Enter rows and columns: 4 4
Enter the matrix:
4 7 1 6
5 7 3 9
3 2 1 2
7 1 6 3
Enter target cost: 25
Number of paths with cost 25 = 2
```

CODE ANALYSIS

Villaruel

- *We solved this problem using recursion by starting at the last cell and working backward to the first cell, since each cell can only be reached from the cell above or the cell to its left. We tested it on the 4 x 4 matrix with a cost of 25 and got 2 paths, which matches the example. The program works well for small matrices, but it is slow for bigger ones because it repeats the same calculations, so we could use memoization to make it faster.*

Cruz

- *For this problem, we used recursion by starting at the last cell and working backward to the first cell, given that each cell can only be reached from the cell directly above or to the left. Testing the approach on a 4×4 matrix with a cost of 25 yielded 2 paths, matching the example. While the program performs well on small matrices, it slows down significantly on larger ones due to redundant calculations, meaning memoization could be applied to improve efficiency.*

CONCLUSION

Villaruel

- *For Hands-on activity #12, me and my partner followed the procedure by reviewing the algorithmic strategies, analyzing the minimum steps code, and rewriting it with memoization to avoid repeating calculations. Working with my partner on the supplementary activity, we solved the matrix path-counting problem using recursion and I saw how memoization could speed it up. It was nice to understand the previous topics and tackle them again at a deeper level, because recursion, brute force, and dynamic programming became strategies I could actually apply and compare.*

Cruz

- *For this activity, we reviewed different algorithmic strategies and matched them to the algorithms from our previous tasks. Through our analysis, we saw why the greedy approach fails, taking 4 steps instead of the optimal 3 for $n = 10$. Then, we rewrote the code with memoization so each subproblem was solved only once, and compared it with the bottom-up version. For the supplementary activity, we solved the matrix path-counting problem using recursion, and by working through the 4×4 example. Importantly, revisiting these previous topics at a deeper level made the methods of recursion, brute force, and dynamic programming strategies much easier for us to actually apply and compare.*

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction, code assistance, explanation of concepts*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: *grammar correction, code assistance, explanation of concepts*

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.